HUFFMAN

```
#include <iostream>
#include <queue>
#include <unordered_map>
using namespace std;

// Node structure
struct Node {
   char ch;
   int freq;
   Node *left, *right;

   Node(char c, int f) {
      ch = c;
      freq = f;
      left = right = NULL;
   }
};

// Comparator for priority queue (min-heap)
struct compare {
   bool operator()(Node* l, Node* r) {
      return l->freq > r->freq;
   }
};

// Function to print Huffman Codes
```

```
void printCodes(Node* root, string code) {
   if (!root)
      return;

   // If leaf node
   if (root->ch != '$')
      cout << root->ch << " : " << code << endl;

   printCodes(root->left, code + "0");
   printCodes(root->right, code + "1");
}

// Main Huffman function
void HuffmanCoding(char chars[], int freq[], int n) {

   priority_queue<Node*, vector<Node*>, compare> pq;

   // Create leaf nodes and push into queue
   for (int i = 0; i < n; i++) {
      pq.push(new Node(chars[i], freq[i]));
   }

   // Build Huffman Tree
   while (pq.size() > 1) {
      Node *left = pq.top(); pq.pop();
      Node *right = pq.top(); pq.pop();

      Node *newNode = new Node('$', left->freq + right->freq);
```

```
        newNode->left = left;
        newNode->right = right;

        pq.push(newNode);
    }

    // Print codes
    printCodes(pq.top(), "");
}

int main() {
    char chars[] = {'A', 'B', 'C', 'D', 'E', 'F'};
    int freq[]  = {5, 9, 12, 13, 16, 45};

    int n = sizeof(chars) / sizeof(chars[0]);

    cout << "Huffman Codes:\n";
    HuffmanCoding(chars, freq, n);

    return 0;
}
```

Output

```
Huffman Codes:
F : 0
C : 100
D : 101
A : 1100
B : 1101
E : 111


=== Code Execution Successful ===
```